

Lab10 Webclient

673380510-1 จิรัชญา เป้าจันทิก Section 1

1. Screenshot ทดสอบทุก endpoint

GET /products/1

lab10 > New Request

GET http://localhost:8080/products/1

Send

Docs Params Authorization Headers 6 Body ● Scripts Settings Cookies

Body Cookies Headers 2 Test Results 200 OK • 180 ms • 245 B Save Response

{ } JSON Preview Visualize

Filter using JSONPath: \$.story

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673380510-1 SEC 1)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

GET /products

lab10 > New Request

GET http://localhost:8080/products

Send

Docs Params Authorization Headers 6 Body ● Scripts Settings Cookies

Body Cookies Headers 2 Test Results 200 OK

{ } JSON Preview Visualize

Filter using JSONPath: \$.story

```
2 {
3   {
4     "id": "1",
5     "name": "iPhone 15 Pro (673380510-1 SEC 1)",
6     "category": "Electronics",
7     "brand": "Apple",
8     "stock": 50,
9     "price": 39900.0,
10    "discountType": "MEMBER",
11    "discountedPrice": 35910.0
12  },
13  {
14    "id": "2",
15    "name": "MacBook Air M3",
16    "category": "Electronics",
17    "brand": "Apple",
18    "stock": 20,
19    "price": 49900.0,
20    "discountType": "NONE",
21    "discountedPrice": 49900.0
22  },
23  {
24    "id": "3",
25    "name": "Samsung Galaxy S24",
26    "category": "Electronics",
27    "brand": "Samsung",
28    "stock": 30,
29    "price": 29900.0,
30    "discountType": "SEASONAL",
31    "discountedPrice": 23920.0
32  }
33 }
```

POST /products

lab10 > New Request

POST

http://localhost:8080/products

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "name": "AirPods Pro 2",
3   "category": "Accessories",
4   "brand": "Apple",
5   "stock": 15,
6   "price": 8990.0,
7   "discountType": "MEMBER"
8 }
```

Body

200 OK • 129 ms • 258 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "id": "567296e2-0326-4079-921a-01a9777958d1",
3   "name": "AirPods Pro 2",
4   "category": "Accessories",
5   "brand": "Apple",
6   "stock": 15,
7   "price": 8990.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 8091.0
10 }
```

lab10 > New Request

GET

http://localhost:8080/products

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Body

200 OK • 31 ms • 751 B

Save Response

JSON

Preview

Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380510-1 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920
3	567296e2-0326-4079-921a-01a9777958d1	AirPods Pro 2	Accessories	Apple	15	8990	MEMBER	8091

DELETE /products/{id}

lab10 > New Request

Save | Share

DELETE

http://localhost:8080/products/567296e2-0326-4079-921a-01a9777958d1

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 1

Test Results

200 OK • 18 ms • 40 B • | Save Response

Raw

Preview

Visualize

1

lab10 > New Request

Save | Share

GET

http://localhost:8080/products

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Body • | 200 OK • 23 ms • 566 B • | Save Response

{ } JSON

Preview

Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380510-1 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

GET /products/1/price

lab10 > New Request

Save | Share

GET

http://localhost:8080/products/1/price

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Body • | 200 OK • 24 ms • 79 B • | Save Response

{ } JSON

Preview

Visualize

Filter using JSONPath: \$.story

1 35910.0

GET /products/category/{cat}

Accessories

lab10 > New Request

SaveShare

GET

http://localhost:8080/products/category/Accessories

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Body

200 OK8 ms267 B

Save Response

JSON

Preview

Visualize

Filter using JSONPath: \$.story

```
1  [
2    {
3      "id": "567296e2-0326-4079-921a-01a9777958d1",
4      "name": "AirPods Pro 2",
5      "category": "Accessories",
6      "brand": "Apple",
7      "stock": 15,
8      "price": 8990.0,
9      "discountType": "MEMBER",
10     "discountedPrice": 8091.0
11   }
12 ]
```

Electronics

lab10 > New Request

SaveShare

GET

http://localhost:8080/products/category/Electronics

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Body

200 OK13 ms566 B

Save Response

JSON

Preview

Visualize

Filter using JSONPath: \$.story

```
2    {
3      "id": "1",
4      "name": "iPhone 15 Pro (673380510-1 SEC 1)",
5      "category": "Electronics",
6      "brand": "Apple",
7      "stock": 50,
8      "price": 39900.0,
9      "discountType": "MEMBER",
10     "discountedPrice": 35910.0
11   },
12   {
13     "id": "2",
14     "name": "MacBook Air M3",
15     "category": "Electronics",
16     "brand": "Apple",
17     "stock": 20,
18     "price": 49900.0,
19     "discountType": "NONE",
20     "discountedPrice": 49900.0
21   },
22   {
23     "id": "3",
24     "name": "Samsung Galaxy S24",
25     "category": "Electronics",
26     "brand": "Samsung",
27     "stock": 30,
28     "price": 29900.0,
29     "discountType": "SEASONAL",
30     "discountedPrice": 23920.0
31   }
32 ]
```

2. ตอบคำถาม

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

Ans **Blocking** เป็นการทำงานที่โปรแกรมต้องรอให้งานหนึ่งเสร็จก่อนจึงจะทำงานต่อ เช่น รอข้อมูลจาก API หรือฐานข้อมูล ทำให้ Thread ถูกใช้งานเพื่อรอผลอยู่ในช่วงเวลานั้น

Reactive Programming เป็นแนวทางแบบ Non-blocking ที่ไม่จำเป็นต้องหยุด Thread เพื่อรอข้อมูล แต่จะทำงานต่อเมื่อข้อมูลพร้อมผ่าน Reactive Stream จึงช่วยให้ระบบรองรับงานหลาย Request ได้อย่างมีประสิทธิภาพมากขึ้น โดยใน Lab นี้ใช้ Mono และ Flux และหลีกเลี่ยงการใช้ .block()

2. Mono vs Flux — ใช้กรณีไหน และทำไม

Ans **Mono<T>** ใช้สำหรับข้อมูลที่มี 0 หรือ 1 ค่า เหมาะกับงานที่คาดว่าจะได้ผลลัพธ์เพียงรายการเดียว เช่น ค้นหา Product ตาม id, บันทึกสินค้า หรือคำนวณราคาหลังส่วนลด

Flux<T> ใช้สำหรับข้อมูลที่มี 0 ถึงหลายค่า เหมาะกับงานที่สามารถคืนผลลัพธ์หลายรายการ เช่น ดึง Product ทั้งหมด หรือค้นหาสินค้าตาม category

ดังนั้น การเลือกใช้ Mono หรือ Flux ขึ้นอยู่กับจำนวนข้อมูลที่ endpoint นั้นสามารถส่งกลับ

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

Ans WebClient ใช้สำหรับส่ง HTTP Request แบบ Reactive และ Non-blocking โดยสามารถเรียก method ต่อกันเป็น chain ได้ ดังนี้

- **.get()** — กำหนด HTTP Method เป็น GET
- **.uri()** — กำหนด URL หรือ endpoint ที่ต้องการเรียก
- **.retrieve()** — ส่ง Request และเตรียมรับ Response
- **.bodyToMono()** — แปลง Response Body เป็น Mono<T> เมื่อคาดว่าจะได้รับ 0 หรือ 1 ค่า
- **.bodyToFlux()** — แปลง Response Body เป็น Flux<T> เมื่อคาดว่าจะได้รับ 0 ถึงหลายค่า

ตัวอย่างเช่น getProductById() ใช้ .get() เลือก GET Request ต่อมา .uri() ระบุ endpoint และต่อมา .retrieve() รับ Response และต่อมา .bodyToMono(Product.class) แปลง Response เป็น Mono<Product> โดยไม่ต้องใช้ .block() รอผล

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository

Java

```
package com.example.lab10.repository;

import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * ProductRepository — In-memory Reactive Repository
 *
 * 

- ✓ โครงสร้างและ annotation ครบแล้ว
- ✗ TODO: เติม method body ให้ครบทุก method


 *
 * ใช้ ConcurrentHashMap เป็น in-memory storage
 * (ไม่ต่อ Database — เน้นฝึก Mono/Flux)
 *
 * Hint:
 * 

- Mono.just(value) คั้นค่าเดียว
- Mono.empty() คั้นเปล่า
- Flux.fromIterable(list) คั้นหลายค่าจาก collection


 */
public class ProductRepository {

    // — In-memory storage —————
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // — Constructor: ใส่ข้อมูลตัวอย่าง —————
    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (673380510-1 SEC 1)",
            "Electronics", "Apple", 50, 39900.0, "MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3",
            "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24",
            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    // — 1. หา Product 1 รายการ —————
    /**
```

```

* TODO: คืน Mono<Product> จาก store โดยใช้ id
*   ถ้าไม่พบให้คืน Mono.empty()
*
* Hint: store.get(id) คืน Product หรือ null
*   ถ้า null ให้ใช้ Mono.empty()
*   ถ้ามีค่าให้ใช้ Mono.just(product)
*/
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    if (product == null) {
        return Mono.empty();
    }

    return Mono.just(product);
}

// — 2. หา Product ทั้งหมด —————
/**
* TODO: คืน Flux<Product> ของทุกรายการใน store
*
* Hint: store.values() คืน Collection<Product>
*   ใช้ Flux.fromIterable(...) แปลงเป็น Flux
*/
public Flux<Product> findAll() {
    // TODO: เติม code ตรงนี้
    return Flux.fromIterable(store.values());
}

// — 3. บันทึก Product —————
/**
* TODO: บันทึก product ลง store แล้วคืน Mono<Product>
*
* Hint: store.put(product.getId(), product)
*   แล้วใช้ Mono.just(product) คืนค่า
*/
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// — 4. ลบ Product —————
/**
* TODO: ลบ product จาก store แล้วคืน Mono<Void>
*

```

```

* Hint: store.remove(id)
*   แล้วใช้ Mono.empty() คืนค่า (Mono<Void>)
*/
public Mono<Void> deleteById(String id) {
    // TODO: เติม code ตรงนี้
    store.remove(id);
    return Mono.empty();
}

// — 5. กรองตาม category —————
/**
* TODO: คืน Flux<Product> ที่ category ตรงกัน
*
* Hint: findAll()
*   .filter(p -> p.getCategory().equalsIgnoreCase(category))
*/
public Flux<Product> findByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return findAll().filter(product -> product.getCategory().equalsIgnoreCase(category));
}
}

```

คำอธิบาย

- ProductRepository ทำหน้าที่จัดเก็บและจัดการข้อมูลสินค้า โดยใน Lab นี้ใช้ ConcurrentHashMap เป็นที่เก็บข้อมูลชั่วคราวแทนการเชื่อมต่อด้านข้อมูล และมีข้อมูลสินค้าเริ่มต้น 3 รายการสำหรับใช้ทดสอบระบบ
- เมธอด findById() ใช้ค้นหาสินค้าจาก id ถ้าพบจะคืนสินค้าในรูปแบบ Mono<Product> แต่ถ้าไม่พบจะคืน Mono.empty() ส่วน findAll() ใช้ Flux.fromIterable() เพื่อคืนสินค้าทั้งหมดในรูปแบบ Flux<Product> ซึ่งเหมาะกับข้อมูลที่มีหลายรายการ
- เมธอด save() ใช้บันทึกสินค้าลงใน store และคืนสินค้าที่บันทึกเป็น Mono<Product> ส่วน deleteById() ใช้ลบสินค้าตาม id และคืน Mono<Void> เนื่องจากไม่มีข้อมูลที่ต้องส่งกลับหลังจากลบ
- สุดท้าย findByCategory() เรียกข้อมูลสินค้าทั้งหมดแล้วใช้ filter() เพื่อเลือกเฉพาะสินค้าที่มี category ตรงกับค่าที่รับเข้ามา โดยใช้ equalsIgnoreCase() เพื่อไม่สนใจตัวพิมพ์เล็ก-ใหญ่

Service

Java

```
package com.example.lab10.service;

import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.UUID;

/**
 * ProductService — Business Logic Layer
 *
 *  @Service, Constructor Injection ครบแล้ว (DIP — SOLID)
 *  TODO: เติม method body ให้ครบทุก method
 *
 * * หน้าที่: รับ request จาก Controller → เรียก Repository → คืนผล
 * * (SRP — แต่ละ class มีหน้าที่เดียว)
 *
 * * Hint Operators ที่ควรใช้:
 * * .map(p -> ...)      แปลงค่า
 * * .flatMap(p -> ...)   async transform
 * * .defaultIfEmpty(...) fallback ถ้าว่าง
 * * .switchIfEmpty(Mono...) fallback Mono ถ้าว่าง
 */
@Service
public class ProductService {

    // — Constructor Injection (DIP — SOLID) —————
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // — 1. ดึง Product 1 รายการ —————
    /**
     * TODO: เรียก repository.findById(id) แล้วคืนผล
     * * ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
     *
     * * Hint: repository.findById(id)
     * * .switchIfEmpty(Mono.error(new RuntimeException(...)))
     */
}
```

```

*/
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(Mono.error(
            new RuntimeException("Product not found: " + id)
        ));
}

```

```

// — 2. ดึง Product ทั้งหมด —————
/**
 * TODO: เรียก repository.findAll() แล้วคืนผล
 */
public Flux<Product> getAll() {
    // TODO: เติม code ตรงนี้
    return repository.findAll(); // ← แก้อบรหัตนี้
}

```

```

// — 3. บันทึก Product —————
/**
 * TODO: เรียก repository.save(product) แล้วคืนผล
 *
 * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่
 * Hint: java.util.UUID.randomUUID().toString()
 */
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }

    return repository.save(product);
}

```

```

// — 4. ลบ Product —————
/**
 * TODO: เรียก repository.deleteById(id) แล้วคืนผล
 */
public Mono<Void> delete(String id) {
    // TODO: เติม code ตรงนี้
    return repository.deleteById(id);
}

```

```

// — 5. กรองตาม category —————
/**
 * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
 */

```

```

public Flux<Product> getByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาหลังส่วนลด —————
/**
 * TODO: หา Product จาก id แล้วคืน discountedPrice
 *
 * Hint: getById(id)
 *       .map(p -> p.getDiscountedPrice())
 */
public Mono<Double> getDiscountedPrice(String id) {
    // TODO: เติม code ตรงนี้
    return getById(id).map(Product::getDiscountedPrice);
}
}

```

คำอธิบาย

- ProductService เป็นส่วนที่จัดการ Business Logic ของระบบ อยู่ระหว่าง Controller และ Repository โดยใช้ Constructor Injection เพื่อรับ ProductRepository เข้ามาใช้งาน
- getById() เรียก Repository เพื่อค้นหาสินค้าตาม id และใช้ switchIfEmpty() เพื่อสร้าง error กรณีไม่พบสินค้า ส่วน getAll() ใช้ดึงสินค้าทั้งหมดจาก Repository โดยตรง
- save() ตรวจสอบก่อนว่าสินค้ามี id หรือไม่ หากไม่มีจะสร้าง id ใหม่ด้วย UUID แล้วจึงส่งไปบันทึกที่ Repository ส่วน delete() ใช้ส่ง id ไปยัง Repository เพื่อลบสินค้า และ getByCategory() ใช้เรียกข้อมูลสินค้าที่ผ่านการกรองตาม category
- สำหรับ getDiscountedPrice() จะค้นหาสินค้าจาก id ก่อน แล้วใช้ map() แปลงข้อมูลจาก Product ให้เหลือเฉพาะราคาหลังส่วนลดในรูปแบบ Mono<Double> โดยยังคงทำงานตามแนวทาง Reactive Programming

Controller

Java

```
package com.example.lab10.controller;

import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

/**
 * ProductController — Reactive REST Controller
 *
 * 

- ✓ @RestController, @RequestMapping, Constructor Injection ครบแล้ว
- ✓ endpoint GET /products/{id} ทำเสร็จแล้วเป็นตัวอย่าง (30%)
- ✗ TODO: เติม method body ของ endpoint ที่เหลือ (70%)


 *
 * Endpoints ที่ต้องทำทั้งหมด:
 * GET /products → Flux<Product> (ดึงทั้งหมด)
 * GET /products/{id} → Mono<Product> ✓ ตัวอย่างทำแล้ว
 * POST /products → Mono<Product> (บันทึก)
 * DELETE /products/{id} → Mono<Void> (ลบ)
 * GET /products/category/{cat} → Flux<Product> (กรอง)
 * GET /products/{id}/price → Mono<Double> (ราคาหลังลด)
 */
@RestController
@RequestMapping("/products")
public class ProductController {

    // — Constructor Injection (DIP — SOLID) —————
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    // =====
    // ✓ ตัวอย่างที่ทำเสร็จแล้ว — ศึกษาแล้วทำ endpoint ที่เหลือ
    // =====

    /**
     * GET /products/{id}
     * คืน Mono<Product> — ค้นหา Product 1 รายการ
     *
     * ทดสอบ: GET http://localhost:8080/products/1
     */
}
```

```
*/
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}
```

```
// =====
// ✖ TODO: เติม method body ด้านล่างนี้
// =====
```

```
/**
 * GET /products
 * TODO: คืน Flux<Product> ทุกรายการ
 *
 * Hint: เรียก service.getAll()
 * ทดสอบ: GET http://localhost:8080/products
 */
```

```
@GetMapping
public Flux<Product> getAll() {
    // TODO: เติม code ตรงนี้
    return service.getAll(); // ← แก่บรรทัดนี้
}
```

```
/**
 * POST /products
 * TODO: รับ Product จาก request body แล้วบันทึก
 *
 * Hint: เรียก service.save(product)
 * ทดสอบ: POST http://localhost:8080/products
 *   Body: { "name": "...", "price": 999.0, ... }
 */
```

```
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // TODO: เติม code ตรงนี้
    return service.save(product); // ← แก่บรรทัดนี้
}
```

```
/**
 * DELETE /products/{id}
 * TODO: ลบ Product และคืน Mono<Void>
 *
 * Hint: เรียก service.delete(id)
 * ทดสอบ: DELETE http://localhost:8080/products/1
 */
```

```
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
```

```

// TODO: เติม code ตรงนี้
return service.delete(id); // ← แก่บรรทัดนี้
}
/**
 * GET /products/category/{category}
 * TODO: คืน Flux<Product> ที่กรองตาม category
 *
 * Hint: เรียก service.getByCategory(category)
 * ทดสอบ: GET http://localhost:8080/products/category/Electronics
 */
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    // TODO: เติม code ตรงนี้
    return service.getByCategory(category); // ← แก่บรรทัดนี้
}
/**
 * GET /products/{id}/price
 * TODO: คืน Mono<Double> ราคาหลังส่วนลด
 *
 * Hint: เรียก service.getDiscountedPrice(id)
 * ทดสอบ: GET http://localhost:8080/products/1/price
 */
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.getDiscountedPrice(id); // ← แก่บรรทัดนี้
}
}
}

```

คำอธิบาย

- ProductController ทำหน้าที่รับ HTTP Request จากผู้ใช้และเรียกใช้ ProductService โดยกำหนด path หลักเป็น /products และคืนข้อมูลในรูปแบบ Mono หรือ Flux ตามจำนวนข้อมูลที่แต่ละ endpoint ต้องส่งกลับ
- ระบบมีทั้งหมด 6 endpoints ได้แก่ GET /products/{id} สำหรับค้นหาสินค้า 1 รายการ, GET /products สำหรับดูสินค้าทั้งหมด และ POST /products สำหรับรับข้อมูล Product จาก Request Body แล้วส่งไปบันทึกผ่าน Service
- นอกจากนี้ DELETE /products/{id} ใช้ลบสินค้าตาม id, GET /products/category/{category} ใช้ค้นหาสินค้าตาม category และ GET /products/{id}/price ใช้ดูราคาหลังส่วนลดของสินค้าที่กำหนด
- โดย Controller ไม่มีการเขียน Business Logic ซ้ำ แต่มีหน้าที่รับ Request แล้วส่งต่อให้ Service จัดการ ทำให้โค้ดแยกหน้าที่ชัดเจนและดูแลแก้ไขได้ง่าย